UCLouvain

epl

École polytechnique de Louvain

Direct device access from the smartNIC towards datacenters disaggregation

Optional subtitle

Author: **Nicolas JEANMENNE**
Supervisor: **Tom BARBETTE**
Readers: **Nikita TYUNYAYEV, Vany INGEZI**
Academic year 2025–2026
Master [120] in Computer Science

Abstract

TO-DO: complete abstract

Contents

Abstract **1**

1 Introduction **1**

2 Background **2**
2.1 Overview of concepts and definitions 2
2.1.1 Storage medium and interface 2
Flash memory behavior 3
PCI Express and NVMe 3
2.1.2 Network 5
2.1.3 Disaggregation 5
2.1.4 Software stack 5
2.2 Architectural challenges 5
2.2.1 Monolithic challenges 5
2.2.2 Disaggregation challenges 6
2.3 State-of-the-art designs and solutions 6
2.3.1 Control and data plane separation 6
2.3.2 Hardware-offloaded storage 7
2.3.3 Open challenges 7
Storage stack overheads 7
Hardware constraints in zero-copy 8
Latency trade-offs in storage offloading 8
2.3.4 Work comparison 9
2.4 Objectives 9
2.5 IO-TCP implementation analysis 10
2.5.1 Deprecated parts 10
2.5.2 Host stack 10
2.5.3 NIC stack 10

3 Implementation **11**

4 Evaluation **12**
4.1 Disks performances baseline 13
4.1.1 Throughput 13
Sequential read 14
Sequential write 15
Random read and write 16
4.1.2 Cost per I/O operation 16
4.1.3 SLC cache 18
4.1.4 Latency 18
4.1.5 Multiple cores 18
4.2 NVMe-over-Fabrics performances 18
4.2.1 Throughput 19
4.3 lighttpd performances 19
4.4 IO-TCP performances 20
4.5 Implementation performances 20

5 Conclusion and future work **21**
5.1 Disk performances i
5.1.1 Read operations i
5.1.2 Write operations ii
5.1.3 Perf categories iii

Bibliography **vi**

List of acronyms

AIO Asynchronous Input/Output. 10

API Application Programming Interface. 5

BDEV Block device. 8, 10

CPU Central Processing Unit. 4–6, 8

DA Disaggregated Architecture. 4

DMA Direct Memory Access. 5

DPDK Data Plane Development Kit. 8

DPU Data Processing Unit. 5, 6

FIO Flexible I/O tester. 10, 14

FPGA Field-Programmable Gate Array. 5

GPU Graphics Processing Unit. 6

HCA Host Channel Adapter. 5

HDD Hard Disk Drive. 2

HTTP Hypertext Transfer Protocol. 4, 15

I/O Input/Output. 4, 5, 7, 10, 11

IOPS Input/Output Operations Per Second. 13

LLC Last Level Cache. 4

MA Monolithic Architecture. 3

NIC Network Interface Card. 5, 6, 8

NPF Network Performance Framework. 10

NVMe Non-Volatile Memory Express. 4–8

NVMe-oF NVMe over Fabric. 5, 11

P2PDMA Peer-to-Peer Direct Memory Access. 5–7

PCIe PCI Express. 6, 8, 10, 11, 13

pSLC Pseudo-Single-Level Cell. 13

RDMA Remote Direct Memory Access. 4, 14

RoCE RDMA over Convergent Ethernet. 4

RTT Round-Trip Time. 4

SPDK Storage Performance Development Kit. 6–8, 10–13

SSD Solid State Drive. 2

TCP Transmission Control Protocol. 4, 6, 14

TLS Transport Layer Security. 4

List of Listings

Chapter 1

Introduction

TODO NT *This is important, spend a bit of time on that*

Chapter 2

Background

This chapter provides an overview of disaggregated datacenter architectures and smartNIC-to-storage interactions. We will first give an overview of the main concepts and definitions used in the literature, then we will review the main architectural challenges and solutions proposed by researchers. Finally, we will list some unresolved challenges that we aim to solve in this thesis.

NT *What is a SmartNIC, how does it fit in a disagragated architecture?*

2.1 Overview of concepts and definitions

High-performance disaggregated storage rely on a tightly coupled pipeline bridging storage media, network interfaces and protocols, and specialized software stacks. To understand the challenges and solutions proposed in the literature, it is important to first define the main concepts layer by layer.

2.1.1 Storage medium and interface

A traditional Hard Disk Drive (HDD) is a mechanical device that stores data on spinning magnetic disks. To read or write data, the drive must physically move a mechanical arm, the read/write head, to the correct track and wait for the disk to rotate to the correct position. These mechanical movements, called seek time and rotational latency take each a few milliseconds, which limits the throughput and increases access time. Typical throughput of a HDD is around 100 MB/s and one hundred requests per second. In contrast, a Solid State Drive (SSD) is a device that stores data on non-volatile flash memory and contains no moving parts : accessing data is a purely electronic operation that takes tens of microseconds. Modern SSDs can achieve throughput of several GB/s and hundreds of thousands to millions of requests per second, depending on how those

devices are connected to the system. SSDs are therefore suitable for high-performance applications as they are a thousand to ten thousand times faster than HDDs. [1], [2]

This speedup has a consequence that motivates this thesis : the storage medium is no longer the bottleneck in the data path nor the slowest component in the system. With several millions of requests per second, the machine must process one request every few hundred nanoseconds, and the software layers such as system calls, the kernel block layer, interrupts, . . . become the new bottleneck. Understanding the storage medium and its interface is therefore crucial to design high-performance storage applications.

Flash memory behavior

Flash memory stores data in pages of a few kilobytes, which are grouped into blocks of a few megabytes. A page can be written only once before it must be erased, and an erase operation can only be performed on a whole block, so an SSD cannot overwrite data in place. This behavior is called the *erase-before-write*, and an internal controller is responsible to write data to a new page and erase old pages in the background. This behavior has direct consequences that reappear in chapter 4 : write operations are slower and more complex than read operations, and continuous write throughput can drop if the Single-Level Cell (SLC) cache is exhausted. A SLC cache is a small portion of the SSD that is used to store data in a single-level cell mode, which is faster than TLC or QLC used for the rest of the device. When the SLC cache is full, the SSD must write data directly to the slower TLC/QLC area, which can cause a significant drop in write throughput.

PCI Express and NVMe

An SSD can be connected to a computer through several interfaces, such as SATA, SAS, or PCIe. SATA and SAS are legacy interfaces that were designed for HDDs, and are therefore limited in terms of throughput. Modern systems connect SSDs to the processor over PCI Express (PCIe), a high-speed point-to-point interconnect that allows for low-latency, high-throughput communication between devices .PCIe has evovled into several generations, PCIe 3.0 transfers rougly 1 GB/s per lane, PCIe 4.0 about 2 GB/s per lane, PCIe 5.0 about 4 GB/s per lane, and PCIe 6.0 about 8 GB/s per lane [3], [4]. A typical SSD or Network Interface Card (NIC) uses four lanes (x4) so PCIe 4.0 provides a total throughput of about 8 GB/s. The PCIe generation and lanes count thus determine the hard ceiling a storage device or network interface can achieve. PCIe also allows for Direct Memory Access (DMA) to and from devices, which allows for direct memory access without CPU involvement, further reducing latency and improving throughput.

Non-Volatile Memory Express (NVMe) is a protocol designed for high-speed communication with non-volatile storage devices, such as SSDs. It provides a low-latency, high-throughput interface that allows multiple commands to be processed in parallel,

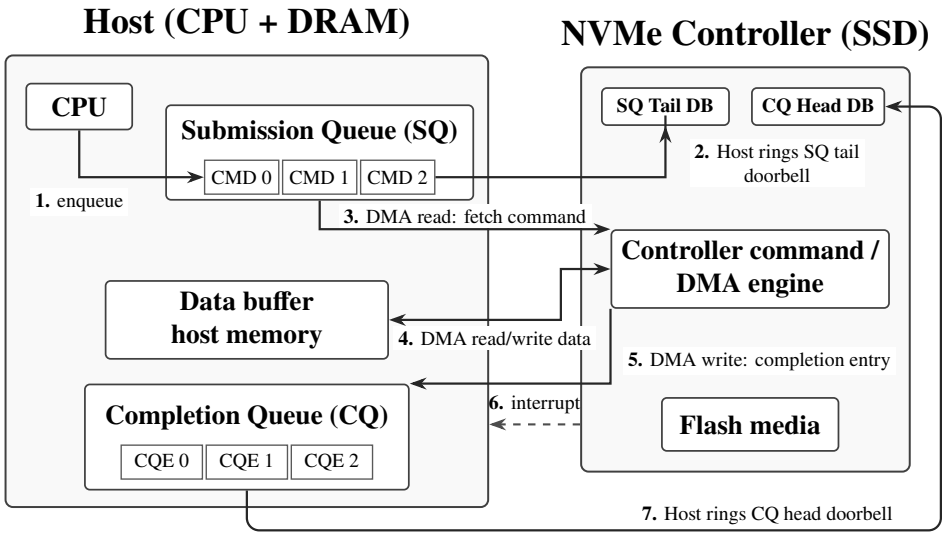


Figure 2.1: Illustration of the NVMe command submission and completion process. The host CPU enqueues commands in the SQ, notifies the controller via a doorbell register, and the controller fetches commands using DMA. Data is transferred between the host and controller using DMA, and upon completion, the controller writes a completion entry to the CQ and signals the host with an interrupt.

making it well-suited for modern storage applications. NVMe uses queues to manage commands and completions. Commands are submitted to the SSD through Submission Queue (SQ)s, the SSD fetches the request from main memory using DMA, processes the request, and places the result in a Completion Queue (CQ), ready to be picked up by the host. Many queues can exist in parallel, and each queue can hold multiple commands. This queuing model is what exposes the internal parallelism of the flash chips to the host. Since a single request takes tens of microseconds, reaching millions of requests per second requires keeping hundreds of requests in flight simultaneously. The number of outstanding requests is called the I/O depth, and it is a key parameter of our evaluation methodology in chapter 4. Figure 2.1 illustrates the NVMe command process.

Finally, NVMe was originally designed for locally attached devices over PCIe. The protocol has been extended to operate over NVMe over Fabric (NVMe-oF), which we discuss in section 2.1.2, as it is central to the SmartNIC architecture studied in this thesis.

2.1.2 Network

2.1.3 Disaggregation

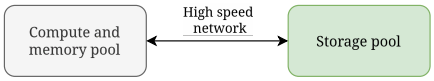


Figure 2.2: Partial disaggregation architecture

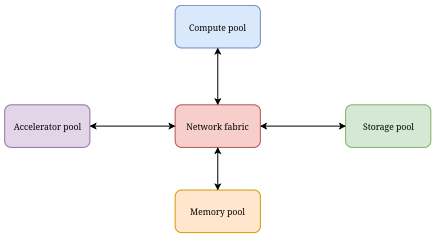


Figure 2.3: Full disaggregation architecture

2.1.4 Software stack

2.2 Architectural challenges

I did not modify this section, you can skip it, I plan to rewrite it later.

Architectural challenges can be grouped into two categories:

- Limitations of traditional monolithic servers, which motivate the transition toward disaggregated architectures.
- New challenges introduced by disaggregation, which break assumptions commonly made in earlier systems research and require new design approaches.

NT *This is too short and it lacks paper citations*

2.2.1 Monolithic challenges

The main limitation of Monolithic Architecture (MA) is **resource stranding**. When one resource is exhausted (for example, memory), other resources on the same server may remain underutilized, leading to waste.

Another important issue in Monolithic Architecture (MA) is availability under failures. A single hardware failure can impact the whole server and make it unavailable.

Finally, hardware replacement and upgrades in Monolithic Architecture (MA) often require costly maintenance operations.

2.2.2 Disaggregation challenges

In disaggregated architecture (Disaggregated Architecture (DA)), the network becomes the central communication path between resource clusters. This creates strict latency and bandwidth requirements: CPU-storage communication typically tolerates millisecond-scale delays, whereas CPU-memory communication requires nanosecond-scale delays (around 100 ns) [5].

Current network stacks also expose strong flexibility-performance trade-offs. NT *Explain a bit what is RDMA and what is RoCE* RDMA and RDMA over Convergent Ethernet (RoCE) offer high performance but limited flexibility due to hardware coupling. In addition, RDMA can suffer from head-of-line blocking, congestion, and vendor lock-in because its requirements are not always satisfied by commodity hardware. By contrast, Linux TCP is more flexible NT *This is too vague* but delivers lower performance and less efficient CPU utilization [6].

2.3 State-of-the-art designs and solutions

2.3.1 Control and data plane separation

NT *Discuss how decoupling the stack helps for scalability maybe? Need to fit better in the disagragation discussion* Standard network stacks like Linux TCP force all operations to be executed on a single plane on the host CPU, causing performance bottlenecks and resource contention. NT *sources? Claim should be backed up* To address this, researchers have proposed split stack architectures that separate control and data operations. This decoupling allows stateful control plane operations to be managed by the CPU, while stateless data plane operations can be offloaded to hardware accelerators, enabling zero-copy mechanisms. NT *Aren't we mixing zero-copy and split stack here?* The term *zero-copy* refers to data transfers between devices without requiring data replication by the CPU, thereby avoiding redundant copies and reducing CPU usage.

Recent state-of-the-art research uses this design to solve encountered bottlenecks rather than offloading every operation. A common design is to separate control operations from data operations. This is the case for **IO-TCP** [7], a HTTP / TLS web content delivery system which splits *mTCP* [8], a userspace stack. The control plane is responsible for connection management, congestion control, and retransmission. It runs on the host CPU, while the data plane is responsible for packet preparation and transfer, running on the smartNIC. This design prevents disk I/O pollution in the CPU cache and reduces contention, notably on the Last Level Cache (LLC) miss rate which is improved by 27 %. This allows a single core to saturate several NVMe disks at 77.4 Gbps, whereas Linux and Atlas [9] (a video streaming FreeBSD server) achieve roughly half that throughput even when using 10 cores. Network performance also benefits from reduced RTT and improved throughput.

NT *This could be more high-level, too many words/concepts are not explained*

While IO-TCP addresses host-centric bottlenecks with a specialized userspace implementation, **ZeroNIC** [6] focuses on flexibility and generality for host networking. Rather than modifying a specific stack, ZeroNIC physically separates the control and data planes to achieve end-to-end zero-copy, and can run on host userspace, kernel, or on an accelerator. Prior approaches required strict page alignment, page remapping, or custom Application Programming Interface (API) to avoid data copies. ZeroNIC, on the other hand, achieves zero-copy without these constraints by delegating payload placement to the NIC hardware. Utilizing on-NIC data structures such as memory segments and flow tables, the hardware calculates memory offsets and DMAs payloads directly to applications (which can reside on CPU, DPU, or FPGA). The software control plane only processes splitted headers, maintaining reliable protocol semantics, without ever touching the payload. This approach allows a single core to saturate a 100 Gbps link with a single TCP flow, while using only 17% of a single CPU.

2.3.2 Hardware-offloaded storage

Beyond network stack offloading, zero-copy mechanisms can also be applied to storage operations. Concerning NVMe storage, **Scalio** [10] focused on NVMe-oF target offloading which extends the zero-copy mechanism, for storage applications. Scalio is a scalable disaggregated key-value store that transfers directly to the Data Processing Unit (DPU)'s Host Channel Adapter (HCA) via P2PDMA. This extends zero-copy to entirely bypass the DPU cores for read operations, creating a new kind of zero-copy data path who drops CPU usage to 0% for these operations, and improves speedup over previous solutions like LEED[11] and Ditto[12]. Scalio also implements batched write operations, which retain write operations until an arbitrary threshold is achieved. All these operations are then run consecutively while a group commit ensures atomicity of operations, improving throughput and reducing the accelerator CPU load.

NT *Good*

2.3.3 Open challenges

While state-of-the-art solutions sucessfully mitigate host CPU bottlenecks throught hardware offloading and split-stack architectures, they still face several limitations. These limitations include storage stack limitations, scalability and latency trade-offs.

Storage stack overheads

Designs like IO-TCP decouple the network control and data planes, but still rely on traditional file systems like ext4 for storage operations. By relying on direct I/O on ext4, the host is forced to maintain file system state. This design assume a static environment

where files do not change during content delivery, which is not always the case in real-world scenarios. Any changes made to the file system on the host side are not propagated to the smartNIC stack, leading to potential inconsistencies and stale data being served. Furthermore, the overhead of maintaining this file system cause IO-TCP to underperform compared to Linux TCP for a small number of connections (typically less than 4), where Linux achieves over 1.5 times the throughput of IO-TCP. Briding this gap by replacing kernel-based file systems with user-space storage stacks like SPDK NT *Are you sure that SPDK will necessarly help there? :d* is necessary to fully leverage the benefits of NVMe performance, but remains challenging due to the lack of mature file system support and abstractions in polled-mode user-space environments.

Hardware constraints in zero-copy

NT *This is not clear, they do one queue per connection right? So this limits the number of concurent conncections*

General-purpose end-to-end zero-copy mechanisms, like those implemented in ZeroNIC, successfully bypass the host CPU but are constrained by the hardware capabilities of the underlying network interface. The number of zero-copy flows that can be supported is strictly bound by the number of hardware queues available on the NIC, because payload placement is delegated to the hardware, thus requiring the flow tables and memory segments discussed earlier. Moreover, the prototype implementation of ZeroNIC frequently encountered PCIe bandwidth limitations when transferring data to GPU, shifting the bottleneck from CPU to the PCIe bus.

Latency trade-offs in storage offloading

Extending zero-copy mechanisms to storage target operations introduces severe latency penalties when attempting to scale throughput. Scalio demonstrates that Peer-to-Peer Direct Memory Access between DPU and NVMe can reduce CPU usage to 0% for read operations, however this is not applicable to write operations. To maintain throughput for write operations, Scalio system must uses batched writes, thus retaining write operations until a threshold is reached before committing them. While this group commit mechanism improves throughput, it also inflates latency. This trade-off between throughput and latency remains a significant challenge in disaggregated storage systems, particularly in environments where low-latency access is critical.

2.3.4 Work comparison

Table 2.1: Comparison of state-of-the-art disaggregated and split-stack networking solutions [6], [7], [10].

Feature	IO-TCP	ZeroNIC	Scalio
System Category	Split-stack TCP	Decoupled host network stack	Hardware-offloaded key-value store
Control Plane Location	Host CPU (runs modified mTCP)	Environment-agnostic (Host kernel, user space, or accelerator)	Client nodes & DPU cores
Data Plane Location	SmartNIC (DPDK userspace)	SmartNIC hardware logic	DPU's HCA
Data Path Mechanism	Command packets, direct disk reads	On-NIC Flow Tables & Memory Segments	P2PDMA direct to NVMe SSDs
Zero-Copy Scope	Host-side zero-copy (intermediate copy required in SmartNIC RAM)	True end-to-end zero-copy (CPU, DPU, GPU, FPGA memory)	True zero-copy for SSD reads (bypasses DPU CPU)
Key Performance Metric	77.4 Gbps on 1 CPU core, 27% LLC miss rate reduction	100 Gbps single-flow TCP at 17% single core usage	0% DPU CPU usage for reads , up to 3.3x speedup over LEED + Ditto
Limitations / Lacks	Relies on ext4 direct I/O rather than SPDK	Zero-copy flow capacity is strictly bound by the hardware's supported NIC queues, prototype faced PCIe bandwidth limits for GPU transfers	Throughput gains from batched writes incur a latency penalty, limiting use in strictly latency-critical environments

2.4 Objectives

NT *Double check that the SPDK implementation is actually done, if not change the text below :d*

This master's thesis focus on partial disaggregation. Our goal is to demonstrate that SPDK can both be used to maximize NVMe throughput capability, and serve as a building block for file systems operations in disaggregated storage solutions. To do so,

we first evaluate the performance of SPDK in a disaggregated architecture, then we adapt the **IO-TCP solution** to use SPDK-based file systems operations instead of direct I/O on a regular ext4 file system. Finally, we evaluate the performance of our solution in terms of throughput and latency, and compare it to existing solutions.

The core contribution is to modify the NIC stack of IO-TCP to use SPDK Block device and blobstore layers to build more optimized file system operations. This allow us to fully exploit PCIe NVMe by bypassing the kernel block layer, with better efficiency in terms of CPU usage and throughput, while maintaining the benefits of a split-stack architecture.

Choosing IO-TCP as a base for our work is motivated by several factors. First, IO-TCP and SPDK operates in userspace, allowing to share memory domains (specifically DPDK hugepages) and avoid kernel context switches. Second, as noted in IO-TCP paper, ext4 is suboptimal for their use case, and that SPDK would be a better alternative. Third, IO-TCP's purpose is disk-bound web content delivery, whereas other solutions like ZeroNIC are general-purpose networking solutions. Finally, SPDK userspace polling mode integrate well with IO-TCP's userspace stack (mTCP), allowing for an unified kernel-bypass architecture.

NT *Should be it's own section, not 2.5*

2.5 IO-TCP implementation analysis

2.5.1 Deprecated parts

2.5.2 Host stack

2.5.3 NIC stack

Chapter 3

Implementation

TODO later

Chapter 4

Evaluation

This chapter will cover evaluation steps-to-step from the baseline to the implemented solution. Information about server specifications (called Frodo) are provided in Table 4.1, and information about the SmartNIC specifications are provided in Table 4.2. The performances evaluation focus mainly on two metrics : throughput and latency. Experiments could be make thanks to Network Performance Framework (NPF) [13].

Table 4.1: Frodo server specifications

OS	Ubuntu 22.04.5 LTS
Kernel	5.15.0-177-generic
CPU	Intel(R) Xeon(R) E-2378 CPU @ 2.60GHz (16 cores)
RAM	2x 16 GB DDR4 3200 MHz
Disks	2x SOLIDIGM SSDPFKKW010X7 (1TB)
NIC	Mellanox ConnectX-6 (cf Table 4.2)

Table 4.2: SmartNIC specification

Model	Mellanox ConnectX-6
OS	Ubuntu 22.04.5 LTS
Kernel	5.15.0-1082-bluefield
CPU	ARM Cortex-A72 (8 cores)
RAM	16 GB DDR4 3200 MHz
NIC Speed	100 Gbps

4.1 Disks performances baseline

The first step in the measurements process is to evaluate the Input/Output (I/O) baseline performances of the disks, with and without SPDK. The first disk is connected to a PCIe 4.0 slot. We will call it disk A. The second disk is connected to a PCIe 3.0 slot. We will call it disk B. We used `Flexible I/O tester (FIO)` [14] with `libaio` [15] engine to test the throughput of the Linux kernel. Then we'll use `SPDK` own BDEV Asynchronous Input/Output (AIO) engine. Specifications of the disks are given in Table 4.3. The disks are SOLIDIGM SSDPFKKW010X7. Vendor theoretical speed are found in [16], [17], [18]. The disks are connected to a PCIe 4.0 x4 and PCIe 3.0 x4 slot respectively. The submission queue is set to 64 bytes and the completion queue is set to 16 bytes. The maximum number of requests is set to 1023. Information about submission queue, completion queue and maximum number of requests have been collected with `nvme-cli` [19].

In the following subsections 4.1.1 and 4.1.4, we will benchmark the throughput and latency of the disks with different file size, block size and I/O depth. This will give us a baseline to compare with the implementation of NVMe-oF, SPDK, and finally **IO-TCP**.

Table 4.3: Disk specifications [16], [17], [18], [19]

Model	SOLIDIGM SSDPFKKW010X7 (P44 Pro, 1 TB)
PCIe slot	Disk A: PCIe 4.0 ×4; Disk B: PCIe 3.0 ×4
Sequential read (vendor)	7000 MB/s
Sequential write (vendor)	6500 MB/s
Submission queue entry	64 bytes
Completion queue entry	16 bytes
`nr_requests`	1023

NT *You need to explain stuff more, what is depth, what is block size, how do you expect them to behave, how do they actually behave?* NT *I think overall profiling is missing to understand why one is fater than the other*

4.1.1 Throughput

Throughput has been evaluated on disk A for different file size, block size and I/O depth. I/O queue depth is set to 64. Block size is set to 4kB and 4MB. File size is set to 4MB, 10MB, 100MB and 1GB. For each configuration, we run 5 times the test and we compute the mean, standard deviation, minimum, median and maximum throughput. Tests are time-based over 10 seconds, with a ramp time (warming up) of 3 seconds. The results are summarized in tables 5.1, 5.2, and 5.3, and 5.4. The figures 4.1, 4.2, 4.3 and 4.4

show the throughput of sequential read and write operations with 4kB and 4MB block size respectively. Finally, we ran a time-based test of 300 seconds and ramp time of 30 seconds to evict eventual SLC caching in subsection 4.1.3.

Sequential read

NT *This is very dense, try to make it readable and list the main takeaways at the end*

Figure 4.1 depicts the throughput over 4 kilobytes block size. At small file size (4MB and 10MB), kernel-path engines are bottlenecked and far behind device bandwidth, while SPDK achieves systemically higher throughput. SPDK deliver up to 4.5 times higher throughput than kernel-path engines. The standard deviation of SPDK is also significantly lower than kernel-path engines, ranging from 313.99 kB/s to 10.7 MB/s while kernel-path engines range from 33.50 MB/s to 61.94 MB/s. At larger file size (100MB and 1GB), SPDK is still ahead of kernel-path engines, but the throughput remains 0.8 times NT *(1.8?)* higher. The standard deviation of SPDK is yet again significantly lower than kernel-path engines, ranging from 1.49 MB/s to 12.46 MB/s while kernel-path engines range from 41.52 MB/s to 55.04 MB/s. The results are summarized in table 5.1. The reason why SPDK performs better than kernel-path engines is that SPDK bypasses the kernel and directly accesses the device, while kernel-path engines have to go through the kernel, which adds overhead. NT *You need to better detail the mechanism*

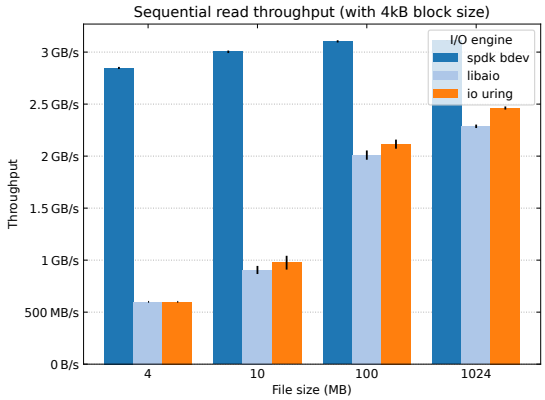


Figure 4.1: Throughput of sequential read operations with 4kB block size

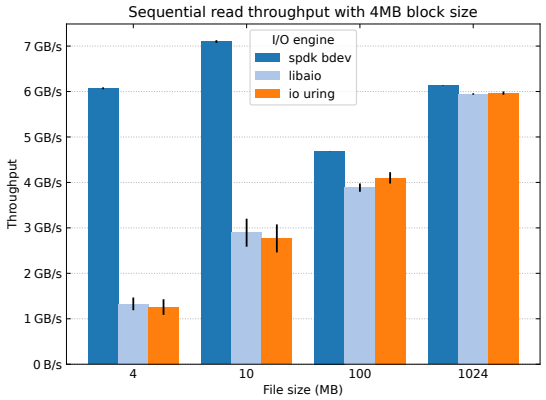


Figure 4.2: Throughput of sequential read operations with 4MB block size

Looking at figure 4.2 which illustrates throughput with 4 megabytes block size, we can observe that througput of all engines increase compared to 4kB block size. SPDK is consistent for all file size, with an average throughput of 6.73 GB/s (maximum achived is 6.75 GB/s). This throughput is close to the vendor theoretical speed of 7 GB/s. Kernel-path engines are still bottlenecked at small file size (4MB and 10MB), with an average throughput of respectively 1.53 GB/s and 2.01 GB/s, thus making SPDK 4.4

times faster than kernel-path engines. At larger file size (100MB and 1GB), kernel-path engines are able to achieve a mean throughput of 3.31 GB/s and 5.43 GB/s respectively, thus making SPDK 1.24 to 2.03 times faster than kernel-path engines. Standard deviation of SPDK is also significantly lower than kernel-path engines, ranging from 11.97 MB/s to 16.74 MB/s while kernel-path engines range from 96.58 MB/s to 218.24 MB/s. The results are summarized in table 5.2.

We can conclude that SPDK is able to achieve higher throughput for sequential read operations with lower standard deviation than kernel-path engines, especially at small file size. It also follows the same order of magnitude for small file size regardless of block size, and is able to achieve close to vendor theoretical speed with large block size.

Sequential write

For sequential write with block size of 4 kB (figure 4.3), we can observe that SPDK is more stable in its mean output, reaching 6.25 GB/s with standard deviation of 35.09 MB/s to 59.09 MB/s. Kernel-path engines are capped at 2.1 GB/s in their best case, with standard deviation of 35.37 MB/s to 353.55 MB/s. Unlike read operations, write operations are bounded by the device NAND program time rather than PCIe bandwidth, so these numbers must also account eventual SLC caching. The results are summarized in table 5.3.

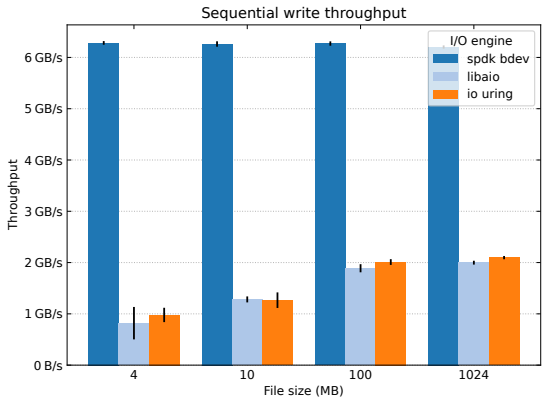


Figure 4.3: Throughput of sequential write operations with 4kB block size

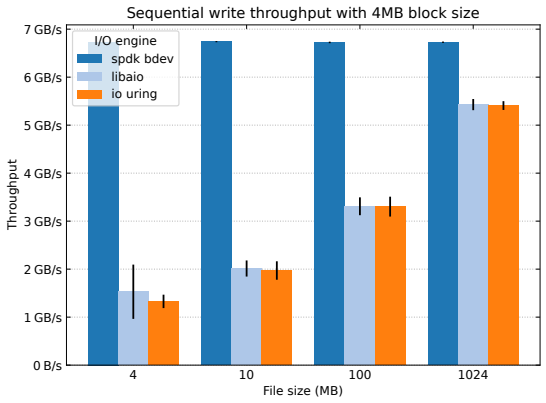


Figure 4.4: Throughput of sequential write operations with 4MB block size

Figure 4.4 shows the throughput of sequential write operations with 4MB block size. At this block size, SPDK has an higher average throughput of 6.73 GB/s with standard deviation of 11.97 MB/s to 16.74 MB/s, while kernel-path engines are capped at 5.43 GB/s with standard deviation of 96.58 MB/s to 595.86 MB/s.

While some configurations are able to exceed the vendor theoretical speed of 6.5 GB/s, caution should be taken as even if not specified by the vendor, the disk can use a dynamic

pSLC write cache to achieve higher throughput. This is why we ran a time-based test of 300 seconds and ramp time of 30 seconds to evict eventual SLC caching in subsection 4.1.3.

Random read and write

TO-DO

4.1.2 Cost per I/O operation

Previous results show that kernel-path engines are impacted by the block size. We can compute the Input/Output Operations Per Second (IOPS) by dividing the throughput by the block size. Below is a table summarizing the average IOPS for previous tests. The results show that SPDK is able to achieve higher IOPS than kernel-path engines, especially at small block size. This is due to the fact that SPDK bypasses the kernel and directly accesses the device, while kernel-path engines have to go through the kernel, which adds overhead.

Table 4.4: Average IOPS for all operations with 4kB and 4MB block size

Block size	**Operations**	`io_uring`	`libaio`	`spdk_bdev`
4kB	Sequential Read	397.03 kIOPS	374.2 kIOPS	1.56 MIOPS
4kB	Random Read	382.62 kIOPS	355.03 kIOPS	742.92 kIOPS
4kB	Sequential Write	397.03 kIOPS	374.2 kIOPS	1.56 MIOPS
4kB	Random Write	361.5 kIOPS	339.55 kIOPS	590.92 kIOPS
4MB	Sequential Read	384.53 IOPS	362.57 IOPS	753.92 kIOPS
4MB	Random Read	822.49 IOPS	834.12 IOPS	1.46 kIOPS
4MB	Sequential Write	750.63 IOPS	767.51 IOPS	1.68 kIOPS
4MB	Random Write	750.63 IOPS	767.51 IOPS	1.68 kIOPS

For example, for sequential read operations with 4kB block size, SPDK achieves on average 753.92 kIOPS while kernel-path engines achieve on average 362.57 kIOPS for `libaio` and 384.53 kIOPS for `io_uring`, nearly 2 times less. For 4MB block size, SPDK got on average 1.5 KIOPS while `io_uring` got 880.83 IOPS and `libaio` got 878.34 IOPS, nearly 1.7 times less.

Figures **??** and **??** show the `perf` of sequential write operations with 4kB block size and 1MB or 4MB file size respectively. Up to 20 % of the CPU time is spent on system calls. Detailed classification of the CPU time is available in table 5.5.

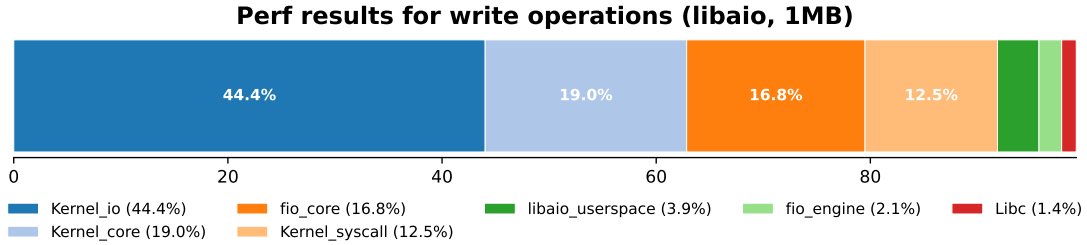


Figure 4.5: Perf of sequential write operations with 4kB block size and 1MB file size with `libaio`

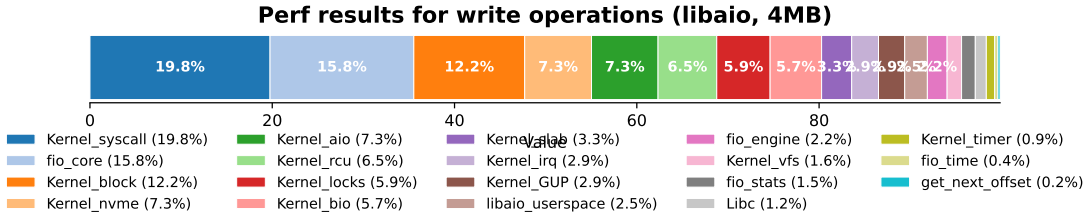


Figure 4.6: Perf of sequential write operations with 4kB block size and 4MB file size with `libaio`

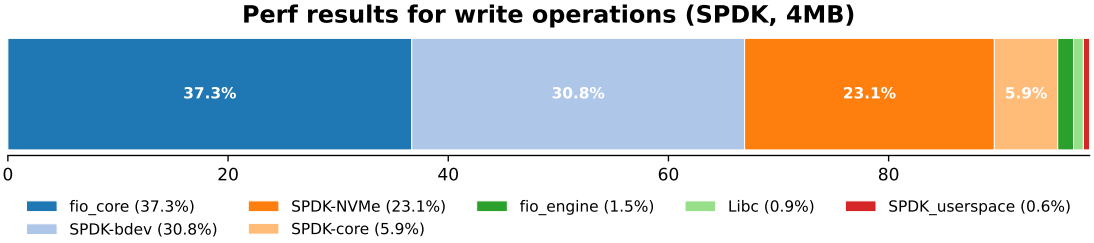


Figure 4.7: Perf of sequential write operations with 4kB block size and 4MB file size with SPDK

4.1.3 SLC cache

TO-DO

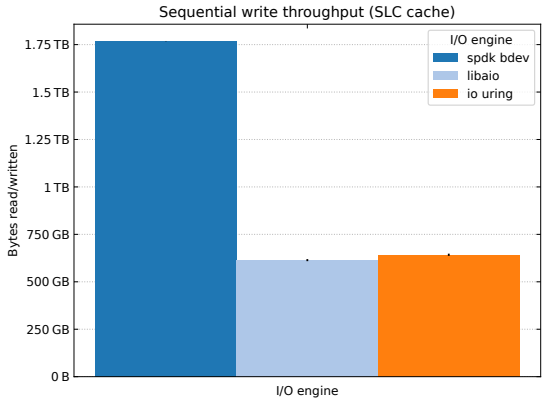


Figure 4.8: Throughput of sequential write operations with 4kB block size and 1MB file size

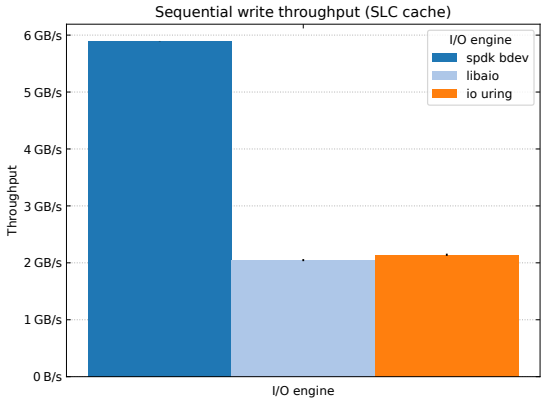


Figure 4.9: Throughput of sequential write operations with 4kB block size and 4MB file size

4.1.4 Latency

TO-DO

NT *Expland the discussion*

4.1.5 Multiple cores

TO-DO

4.2 NVMe-over-Fabrics performances

In this section we evaluate the throughput and latency of NVMe-over-Fabrics with Linux kernel on disk A. The tests evaluate both TCP and RDMA transport. The tests are made with `FIO` [14] with `io_uring` engine. Queue depth is fixed to 64.

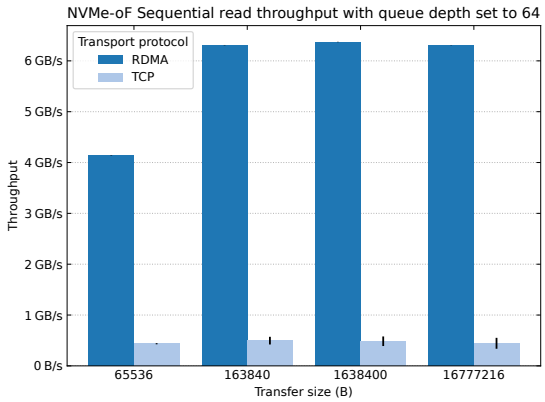


Figure 4.10: Throughput of sequential read operations for NVMe-over-Fabrics with large file size

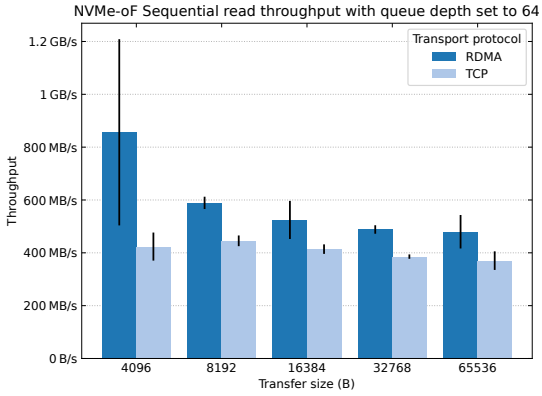


Figure 4.11: Throughput of sequential read operations for NVMe-over-Fabrics with small file size

4.2.1 Throughput

4.3 lighttpd performances

We benchmarked the HTTP server `lighttpd` original version (1.4.32) in order to analyze the difference between the original version and the IO-TCP version. The benchmark is made with `wrk` [20]. To be fair with IO-TCP, we restrict the `lighttpd` to a single core. Figures 4.12 and 4.13 show the throughput of `lighttpd` on Frodo server (x86_64 core) and on SmartNIC (ARM Cortex-A72 core) respectively. We observe that a single core on Frodo is nearly able to saturate the disk bandwidth.

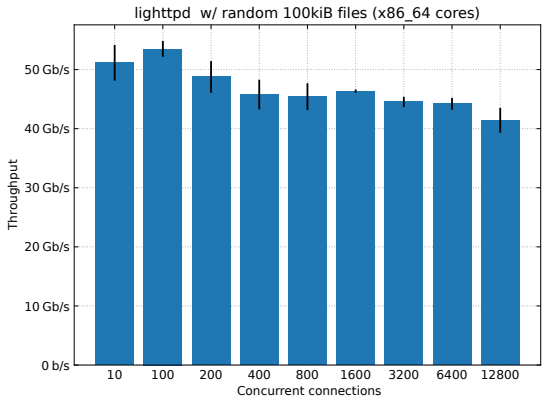


Figure 4.12: Throughput of lighttpd on Frodo server (x86_64 core)

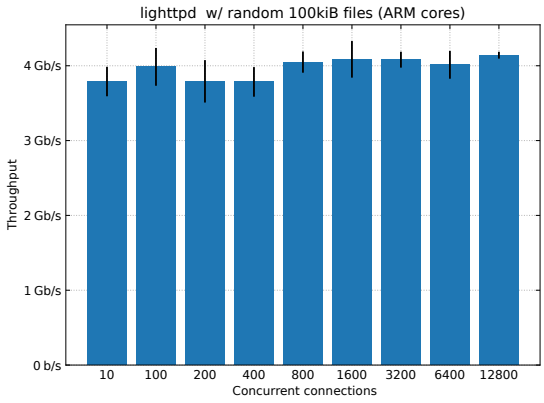


Figure 4.13: Throughput of lighttpd on SmartNIC (ARM Cortex-A72 core)

4.4 IO-TCP performances

4.5 Implementation performances

Chapter 5

Conclusion and future work

TODO

Appendix : additional plots data

5.1 Disk performances

5.1.1 Read operations

Table 5.1: Statistical summary of figure 4.1 : read throughput with 4kB block size

Engine	Size (MB)	Mean	Std. Dev.	Min	Median	Max
io_uring	4	584.21MB/s	33.50MB/s	524.29MB/s	599.19MB/s	599.19MB/s
io_uring	10	956.43MB/s	61.94MB/s	873.81MB/s	953.25MB/s	1.05GB/s
io_uring	100	2.01GB/s	41.52MB/s	1.94GB/s	2.02GB/s	2.06GB/s
io_uring	1024	2.38GB/s	55.04MB/s	2.30GB/s	2.36GB/s	2.45GB/s
libaio	4	584.21MB/s	33.50MB/s	524.29MB/s	599.19MB/s	599.19MB/s
libaio	10	905.59MB/s	43.51MB/s	873.81MB/s	873.81MB/s	953.25MB/s
libaio	100	1.89GB/s	45.43MB/s	1.84GB/s	1.91GB/s	1.94GB/s
libaio	1024	2.25GB/s	13.68MB/s	2.24GB/s	2.25GB/s	2.27GB/s
spdk_bdev	4	2.63GB/s	10.07MB/s	2.62GB/s	2.62GB/s	2.64GB/s
spdk_bdev	10	2.98GB/s	313.99kB/s	2.98GB/s	2.98GB/s	2.98GB/s
spdk_bdev	100	2.51GB/s	1.49MB/s	2.51GB/s	2.51GB/s	2.51GB/s
spdk_bdev	1024	3.00GB/s	12.46MB/s	2.99GB/s	3.01GB/s	3.01GB/s

Table 5.2: Statistical summary of figure 4.2 : read throughput with 4MB block size

Engine	Size (MB)	Mean	Std. Dev.	Min	Median	Max
io_uring	4	1.33GB/s	147.37MB/s	1.05GB/s	1.40GB/s	1.40GB/s
io_uring	10	1.97GB/s	202.60MB/s	1.68GB/s	2.10GB/s	2.10GB/s
io_uring	100	3.30GB/s	218.24MB/s	2.69GB/s	3.38GB/s	3.38GB/s
io_uring	1024	5.41GB/s	96.58MB/s	5.32GB/s	5.36GB/s	5.56GB/s
libaio	4	1.53GB/s	595.86MB/s	262.14MB/s	1.40GB/s	2.10GB/s
libaio	10	2.01GB/s	176.85MB/s	1.68GB/s	2.10GB/s	2.10GB/s
libaio	100	3.31GB/s	196.16MB/s	2.76GB/s	3.38GB/s	3.38GB/s
libaio	1024	5.43GB/s	122.58MB/s	5.24GB/s	5.38GB/s	5.62GB/s
spdk_bdev	4	6.73GB/s	11.97MB/s	6.72GB/s	6.73GB/s	6.75GB/s
spdk_bdev	10	6.74GB/s	12.38MB/s	6.72GB/s	6.74GB/s	6.75GB/s
spdk_bdev	100	6.72GB/s	16.74MB/s	6.70GB/s	6.72GB/s	6.75GB/s
spdk_bdev	1024	6.73GB/s	14.98MB/s	6.71GB/s	6.73GB/s	6.75GB/s

5.1.2 Write operations

Table 5.3: Statistical summary of figure 4.3 : write throughput with 4kB block size

Engine	Size (MB)	Mean	Std. Dev.	Min	Median	Max
io_uring	4	1.05GB/s	0.00B/s	1.05GB/s	1.05GB/s	1.05GB/s
io_uring	10	1.39GB/s	102.56MB/s	1.31GB/s	1.31GB/s	1.50GB/s
io_uring	100	1.71GB/s	165.90MB/s	1.57GB/s	1.61GB/s	1.91GB/s
io_uring	1024	2.10GB/s	55.96MB/s	2.03GB/s	2.08GB/s	2.16GB/s
libaio	4	608.59MB/s	202.29MB/s	246.72MB/s	699.05MB/s	699.05MB/s
libaio	10	991.38MB/s	52.21MB/s	953.25MB/s	953.25MB/s	1.05GB/s
libaio	100	1.87GB/s	87.03MB/s	1.78GB/s	1.84GB/s	1.98GB/s
libaio	1024	2.02GB/s	35.20MB/s	1.98GB/s	2.01GB/s	2.07GB/s
spdk_bdev	4	5.16GB/s	2.48GB/s	731.38MB/s	6.24GB/s	6.34GB/s
spdk_bdev	10	6.28GB/s	62.16MB/s	6.23GB/s	6.24GB/s	6.36GB/s
spdk_bdev	100	6.27GB/s	49.40MB/s	6.23GB/s	6.25GB/s	6.33GB/s
spdk_bdev	1024	6.25GB/s	35.50MB/s	6.19GB/s	6.26GB/s	6.28GB/s

Table 5.4: Statistical summary of figure 4.4 : write throughput with 4MB block size

Engine	Size (MB)	Mean	Std. Dev.	Min	Median	Max
io_uring	4	1.33GB/s	147.37MB/s	1.05GB/s	1.40GB/s	1.40GB/s
io_uring	10	1.97GB/s	202.60MB/s	1.68GB/s	2.10GB/s	2.10GB/s
io_uring	100	3.30GB/s	218.24MB/s	2.69GB/s	3.38GB/s	3.38GB/s
io_uring	1024	5.41GB/s	96.58MB/s	5.32GB/s	5.36GB/s	5.56GB/s
libaio	4	1.53GB/s	595.86MB/s	262.14MB/s	1.40GB/s	2.10GB/s
libaio	10	2.01GB/s	176.85MB/s	1.68GB/s	2.10GB/s	2.10GB/s
libaio	100	3.31GB/s	196.16MB/s	2.76GB/s	3.38GB/s	3.38GB/s
libaio	1024	5.43GB/s	122.58MB/s	5.24GB/s	5.38GB/s	5.62GB/s
spdk_bdev	4	6.73GB/s	11.97MB/s	6.72GB/s	6.73GB/s	6.75GB/s
spdk_bdev	10	6.74GB/s	12.38MB/s	6.72GB/s	6.74GB/s	6.75GB/s
spdk_bdev	100	6.72GB/s	16.74MB/s	6.70GB/s	6.72GB/s	6.75GB/s
spdk_bdev	1024	6.73GB/s	14.98MB/s	6.71GB/s	6.73GB/s	6.75GB/s

5.1.3 Perf categories

Table 5.5: Symbol taxonomy for fio engine comparison

Category	Symbol patterns
fio_engine	`fio_libaio_`, `fio_ioring_`
liburing_userspace	`__io_uring`, `io_uring_get_sqe`, `io_uring_prep_`, `io_uring_submit`, `io_uring_queue`, `io_uring_peek_batch`, `io_uring_wait_cqe`, `io_uring_cqe_seen`
libaio_userspace	`io_queue_init`, `^io_submit$`, `^io_getevents$`, `^io_destroy$`, `^io_setup$`, `^io_cancel$`
fio_stats	`add_slat_sample`, `add_clat_sample`, `add_lat_sample`, `flow_threshold_exceeded`, `in_ramp_period`, `ramp_period_over`, `queue_full`, `add_io_u_sample`, `add_lat_percentile`, `stat_add_sample`
fio_core	`io_u_`, `get_io_u`, `put_io_u`, `td_io_`, `td_set_runstate`, `do_io`, `fill_io_u`, `init_io_u`, `put_file`, `get_file`, `get_next_seq_offset`, `io_completed`, `io_queue_event`, `account_io_completion`, `lock_file`, `log_io_u`, `fio_backend`, `fio_file_done`, `fio_file_open`, `fio_fill_issue_time`, `fio_`, `td_`

Category	Symbol patterns
fio_time	__fio_gettime, fio_gettime, fio_clock_init, fio_get_mono_time, ntime_since, clock_thread_fn
Libc	__GI_, __libc_, __memmove_, __memset_, memcpy, memmove, memset, __clock_
Kernel_syscall	do_syscall_64, syscall_enter_from_user_mode, syscall_exit_to_user_mode, syscall_return_via_sysret, entry_SYSCALL_64, __irqentry_text_start, __irqentry_text_end, __x64_sys_, __do_sys_, __se_sys_, syscall
Kernel_aio	aio_, io_submit_one
Kernel_io_uring	io_uring, io_submit_sqes, io_submit_flush_completions
Kernel_nvme	nvme_, __nvme_
Kernel_block	blk_mq_, __blk_mq_, blk_account_io_, blk_add_, blk_attempt_plug, blk_finish_plug_list, blk_flush_plug_list, blk_update_request, blk_stat_add, blk_queue_, __blk_queue_, blk_start_plug, blk_throtl_bio, blk_try_enter_queue, blk_add_timer, blk_cgroup_bio_start, blkdev_direct_IO, blkdev_write_iter, blkdev_bio_end_io, kthread_blkcg
Kernel_bio	bio_, __bio_, submit_bio_, submit_bio, guard_bio_eod
Kernel_irq	handle_edge_irq, handle_irq_event, __handle_irq_event_percpu, irq_chip_ack_parent, apic_ack_irq, __do_softirq, do_softirq, irq_exit_rcu, irq_enter_rcu
Kernel_locks	_raw_spin_lock, _raw_read_lock, _raw_write_lock, lock_timer_base, _raw_spin_unlock, queued_spin_lock, native_queued_spin
Kernel_rcu	synchronize_rcu, rcu_read_lock, __srcu_read_lock, __srcu_read_unlock, rcu_read_unlock, call_rcu, rcu_do_batch
Kernel_GUP	follow_page, follow_page_pte, pin_user_pages, get_user_pages, __gup, unpin_user_page, __unpin, try_grab_compound_head
Kernel_mm	__get_free_pages, __alloc_pages, free_pages, free_unref_page, rmqueue, free_pcppages_bulk, __mod_lruvec_state, __mod_node_page_state, mark_page_accessed, folio_, __folio
Kernel_timer	hrtimer, __hrtimer, clockevents_, tick_, ktime_get, ktime_, sched_clock, __sched_clock, rcu_sched_clock_irq

Category	Symbol patterns
Kernel_sched	`__schedule`, `schedule`, `finish_task_switch`, `try_to_wake_up`, `select_task_rq`, `ttwu`, `dequeue_task`, `enqueue_task`
Kernel_vfs	`vfs_`, `__vfs_`, `do_iter_read`, `do_iter_write`, `new_sync_`, `kiocb`, `generic_file_`, `iomap_`, `fsnotify`, `inode_`, `filemap_`, `iomap_dio`
Kernel_slab	`kmalloc`, `__kmalloc`, `kfree`, `kmem_cache_`, `__slab_`, `slab_`, `mempool_`, `kmemdup`

Bibliography

[1] R. Mecheloni, A. Marelli, and K. Eshghi, "Inside solid state drives (SSD)," in *Inside Solid State Drives (SSD)*, ser. Springer Series in Advanced Microelectronic, Second Edition, vol. 37, Springer, 2018, pp. 1–41, ISBN: 78-981-13-0598-6. DOI: 10.1007/978-981-13-0599-3 [Online]. Available: https://link.springer.com/book/10.1007/978-981-13-0599-3

[2] G. Haas and V. Leis, "What modern NVMe storage can do, and how to exploit it: High-performance i/o for high-performance storage engines," *Proceedings of the VLDB Endowment*, vol. 16, no. 9, pp. 2090–2102, May 2023, ISSN: 2150-8097. DOI: 10.14778/3598581.3598584 Accessed: Nov. 5, 2026. [Online]. Available: https://dl.acm.org/doi/10.14778/3598581.3598584

[3] "What bit rates does the PCIe 5.0 specification support and how does it compare to prior PCIe generations? | PCI-SIG," Accessed: Nov. 7, 2026. [Online]. Available: https://pcisig.com/what-bit-rates-does-pcie-50-specification-support-and-how-does-it-compare-prior-pcie-generations

[4] "PCI express 6.0 specification | PCI-SIG," Accessed: Jul. 28, 2026. [Online]. Available: https://pcisig.com/pci-express-6.0-specification

[5] R. Lin, Y. Cheng, M. D. Andrade, L. Wosinska, and J. Chen, "Disaggregated data centers: Challenges and trade-offs," *IEEE Communications Magazine*, vol. 58, no. 2, pp. 20–26, Feb. 2020, ISSN: 1558-1896. DOI: 10.1109/MCOM.001.1900612 Accessed: Nov. 2, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/8999422

[6] A. Skiadopoulos et al., "High-throughput and flexible host networking for accelerated computing," presented at the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 405–423, ISBN: 978-1-939133-40-3. Accessed: Oct. 19, 2025. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/skiadopoulos

[7] T. Kim, D. M. Ng, J. Gong, Y. Kwon, M. Yu, and K. Park, "Rearchitecting the TCP stack for i/o-offloaded content delivery," presented at the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), 2023, pp. 275–292,

ISBN: 978-1-939133-33-5. Accessed: Oct. 19, 2025. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/kim-taehyun

[8] E. Jeong et al., "mTCP: A highly scalable user-level TCP stack for multicore systems," presented at the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14), 2014, pp. 489–502, ISBN: 978-1-931971-09-6. Accessed: Nov. 24, 2025. [Online]. Available: https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/jeong

[9] I. Marinos, R. N. Watson, M. Handley, and R. R. Stewart, "Disk|crypt|net: Rethinking the stack for high-performance video streaming," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, Los Angeles CA USA: ACM, Aug. 7, 2017, pp. 211–224, ISBN: 978-1-4503-4653-5. DOI: 10.1145/3098822.3098844 Accessed: Jul. 25, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3098822.3098844

[10] X. Sun, M. Zhang, Y. Shan, K. Chen, J. Jiang, and Y. Wu, "Scalio: Scaling up DPU-based JBOF key-value store with NVMe- oF target offload," presented at the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25), 2025, pp. 449–464, ISBN: 978-1-939133-47-2. Accessed: Oct. 18, 2025. [Online]. Available: https://www.usenix.org/conference/osdi25/presentation/sun

[11] Z. Guo, H. Zhang, C. Zhao, Y. Bai, M. Swift, and M. Liu, "LEED: A low-power, fast persistent key-value store on SmartNIC JBOFs," in *Proceedings of the ACM SIGCOMM 2023 Conference*, ser. ACM SIGCOMM '23, New York, NY, USA: Association for Computing Machinery, Sep. 1, 2023, pp. 1012–1027, ISBN: 979-8-4007-0236-5. DOI: 10.1145/3603269.3604880 Accessed: Jul. 28, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3603269.3604880

[12] J. Shen et al., "Ditto: An elastic and adaptive memory-disaggregated caching system," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23, New York, NY, USA: Association for Computing Machinery, Oct. 23, 2023, pp. 675–691, ISBN: 979-8-4007-0229-7. DOI: 10.1145/3600006.3613144 Accessed: Jul. 28, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3600006.3613144

[13] T. Barbette, *Tbarbette/npf*, Mar. 14, 2026. Accessed: Apr. 25, 2026. [Online]. Available: https://github.com/tbarbette/npf

[14] "Kernel/git/axboe/fio.git - fio - flexible IO tester," Accessed: Apr. 25, 2026. [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/axboe/fio.git/

[15] "Overview - libaio - pagure.io," Accessed: Apr. 24, 2026. [Online]. Available: https://pagure.io/libaio

[16] "Solidigm p44 pro 1 TB specs," TechPowerUp, Accessed: Aug. 5, 2026. [Online]. Available: https://www.techpowerup.com/ssd-specs/solidigm-p44-pro-1-tb.d1124

[17] "SOLIDIGM SSDPFKKW010x7 - benchmark performance," Accessed: Aug. 5, 2026. [Online]. Available: https://www.harddrivebenchmark.net/hdd.php?hdd=SOLIDIGM+SSDPFKKW010X7&id=33003

[18] "SSDPFKKW010x7x1|[performance upgrade] SOLIDIGM p44- PRO SSDPFKKW010x7x1 m.2 PCIE4.0 1tb solid-state drive SSD, unlocking high-speed read and write experience!-HK STES TECHNOLOGY CO., LIMITED," Accessed: Aug. 5, 2026. [Online]. Available: https://www.stesz.com/m2/SSDPFKKW010X7X1-e.html

[19] *Linux-nvme/nvme-cli*, original-date: 2015-02-05T22:59:04Z, Apr. 24, 2026. Accessed: Apr. 26, 2026. [Online]. Available: https://github.com/linux-nvme/nvme-cli

[20] W. Glozer, *Wg/wrk*, original-date: 2012-03-20T11:12:28Z, Aug. 4, 2026. Accessed: Aug. 4, 2026. [Online]. Available: https://github.com/wg/wrk

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
École polytechnique de Louvain
Rue Archimède, 1 bte L6.11.01, 1348 Louvain-la-Neuve, Belgique | **www.uclouvain.be/epl**